

CPRE 4880 - Section A1

MP - 0

Cameron Gilbertson, Matthew Smith, Jessica Reiff, Collin Van Weelden

4 February 2026

Dr. Phillip Jones

TABLE OF CONTENTS

NES Bootloader..... 3

Processing System.....4

Block Design..... 8

Vitis..... 11

Bonus Credit..... 17

Student Effort.....18

NES Bootloader

This lab began with downloading and performing a light analysis of the NES bootloader files provided in the lab handout. A block diagram of the main loop was created, and can be found below in Figure #.

Q: How nes_bootloader.c currently works. Using a similar approach as what is presented in Chapter 1 of the Wolf textbook, draw a high-level structural diagram. How does NESCore_Callback_OutputFrame() get called?

The NESCore_Callback_OutputFrame() is a specific function that gets called when the NES State of the Picture Processing Unit (PPU) is SCAN_UNKNOWN_START and the current frame count is 0. SCAN_UNKNOWN_START indicates that the PPU scanline has not yet been defined. This can occur when the emulator is powering on, resetting, or just out of sync. The NESCore_Callback_OutputFrame() function takes the WorkFrame, trashes it, and flushes the cache. This allows the emulator to stabilize during startup while maintaining timing synchronization.

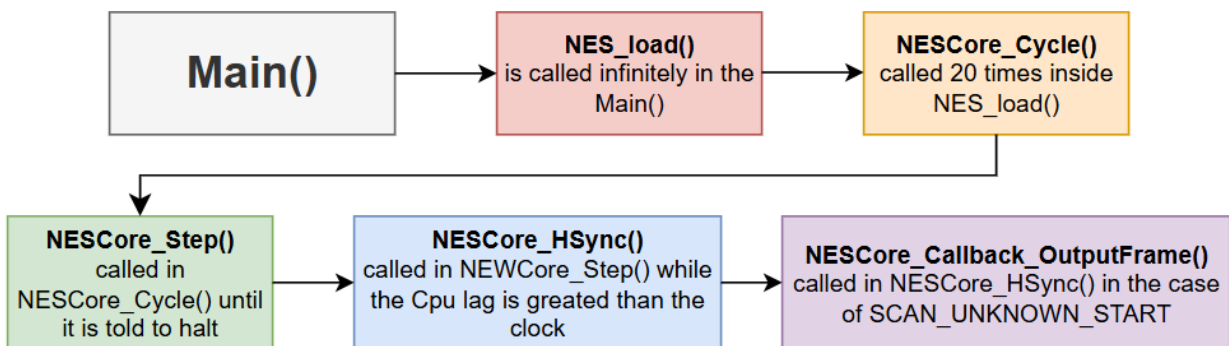


Figure 1: Flow Chart illustrating how the NESCore_Callback_OutputFrame() function is called.

Processing System

At this point in the lab, the Processing System (PS) has been instantiated. The Zynq Block Design is shown in Figure # below.

Q: Click three different green boxes and in your writeup, describe what configuration options are available and how they may be potentially useful in an embedded system.

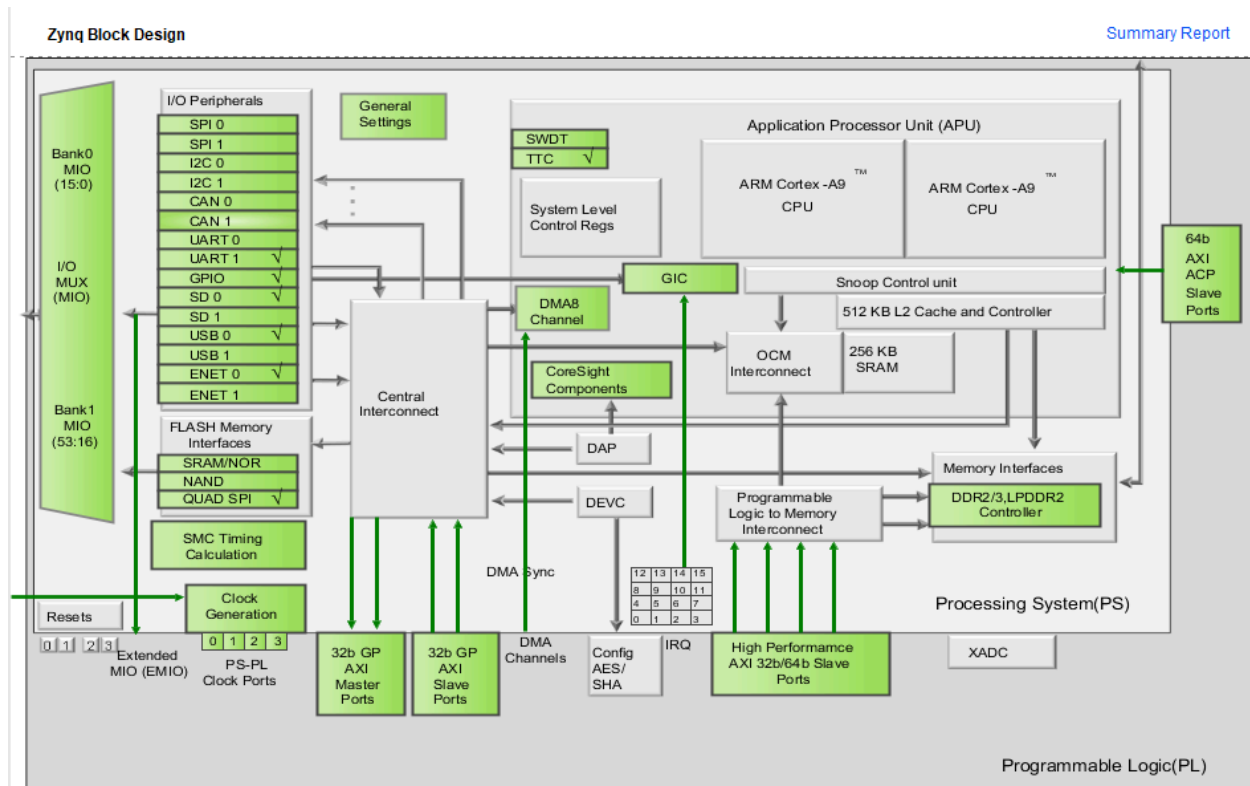


Figure 2: Zynq Block Design of the Processing System.

The green boxes can be selected and display the system's configurable options.

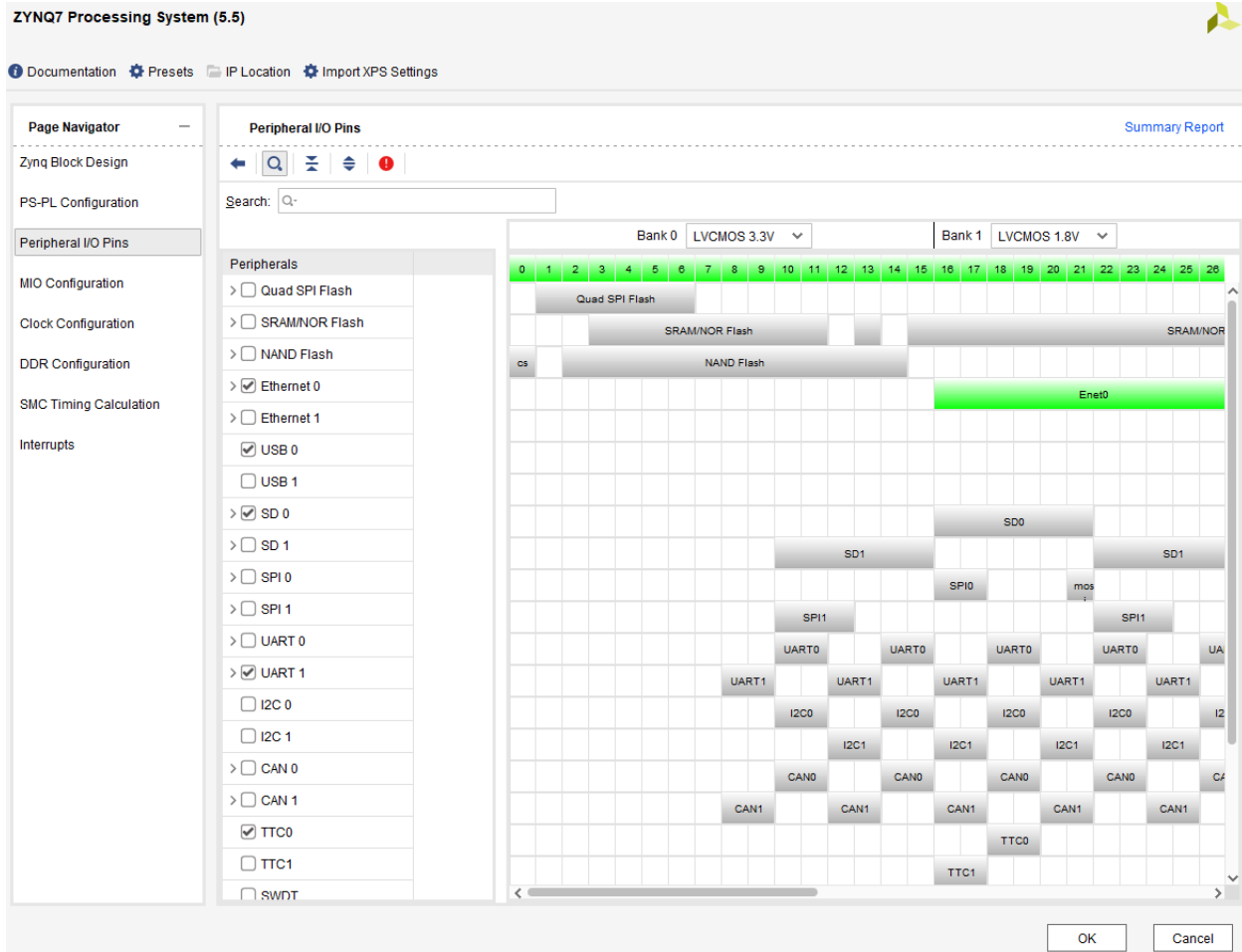


Figure 3: Zync Block Design of the Processing System.

The first option available is the Peripheral I/O Pins which is helpful for being able to enable and know what peripheral I/Os that are available which is always needed when trying to include or communicate to other modules outside the FPGA.

ZYNQ7 Processing System (5.5)

Documentation
Presets
IP Location
Import XPS Settings

Page Navigator

Zynq Block Design
PS-PL Configuration
Peripheral I/O Pins
MIO Configuration
Clock Configuration
DDR Configuration
SMC Timing Calculation
Interrupts

Interrupts

Search: Q-

Interrupt Port	ID	Description
☐ Fabric Interrupts		Enable PL Interrupts to PS and vice versa
PL-PS Interrupt Ports		
☐ IRQ_F2P[15:0]	[91:84], [68:6	Enables 16-bit shared interrupt port from the PL. MSB is assigned the hi
☐ Core0_nFIQ	28	Enables fast private interrupt signal for CPU0 from the PL
☐ Core0_nIRQ	31	Enables private interrupt signal for CPU0 from the PL
☐ Core1_nFIQ	28	Enables fast private interrupt signal for CPU1 from the PL
☐ Core1_nIRQ	31	Enables private interrupt signal for CPU1 from the PL
PS-PL Interrupt Ports		
☐ IRQ_P2F_DMAC_ABORT		Enables shared interrupt abort signal from DMAC to the PL
☐ IRQ_P2F_DMAC0		Enables shared interrupt signal 0 from DMAC to the PL
☐ IRQ_P2F_DMAC1		Enables shared interrupt signal 1 from DMAC to the PL
☐ IRQ_P2F_DMAC2		Enables shared interrupt signal 2 from DMAC to the PL
☐ IRQ_P2F_DMAC3		Enables shared interrupt signal 3 from DMAC to the PL
☐ IRQ_P2F_DMAC4		Enables shared interrupt signal 4 from DMAC to the PL
☐ IRQ_P2F_DMAC5		Enables shared interrupt signal 5 from DMAC to the PL
☐ IRQ_P2F_DMAC6		Enables shared interrupt signal 6 from DMAC to the PL
☐ IRQ_P2F_DMAC7		Enables shared interrupt signal 7 from DMAC to the PL
☐ IRQ_P2F_SMC		Enables shared interrupt signal from SMC to the PL
☐ IRQ_P2F_QSPI		Enables shared interrupt signal from QSPI to the PL
☐ IRQ_P2F_CTI		Enables shared interrupt signal from CTI to the PL
☐ IRQ_P2F_GPIO		Enables shared interrupt signal from GPIO to the PL
☐ IRQ_P2F_USB0		Enables shared interrupt signal from USB 0 to the PL
☐ IRQ_P2F_ENET0, IRQ_P2F_E...		Enables shared interrupt and wake signals from ETHERNET 0 to the PL
☐ IRQ_P2F_SDIO0		Enables shared interrupt signal from SDIO 0 to the PL
☐ IRQ_P2F_I2C0		Enables shared interrupt signal from I2C 0 to the PL
☐ IRQ_P2F_SPI0		Enables shared interrupt signal from SPI0 to the PL
☐ IRQ_P2F_UART0		Enables shared interrupt signal from UART 0 to the PL

Figure 5: Zynq Block Design of the Processing System.

Lastly, we looked at the ability to set interrupts on different ports, in particular for the processing system and the programmable logic pins which is useful when needing to halt the processor for high priority tasks based on a sensor.

Block Design

Q: For the Block Design you just created: Are these buttons, LEDs, and switches connected via the PS subsystem or the PL subsystem? Briefly defend your answer. Note also that all three peripherals appear to be the same exact IP type (axi_gpio) – how can this be possible?

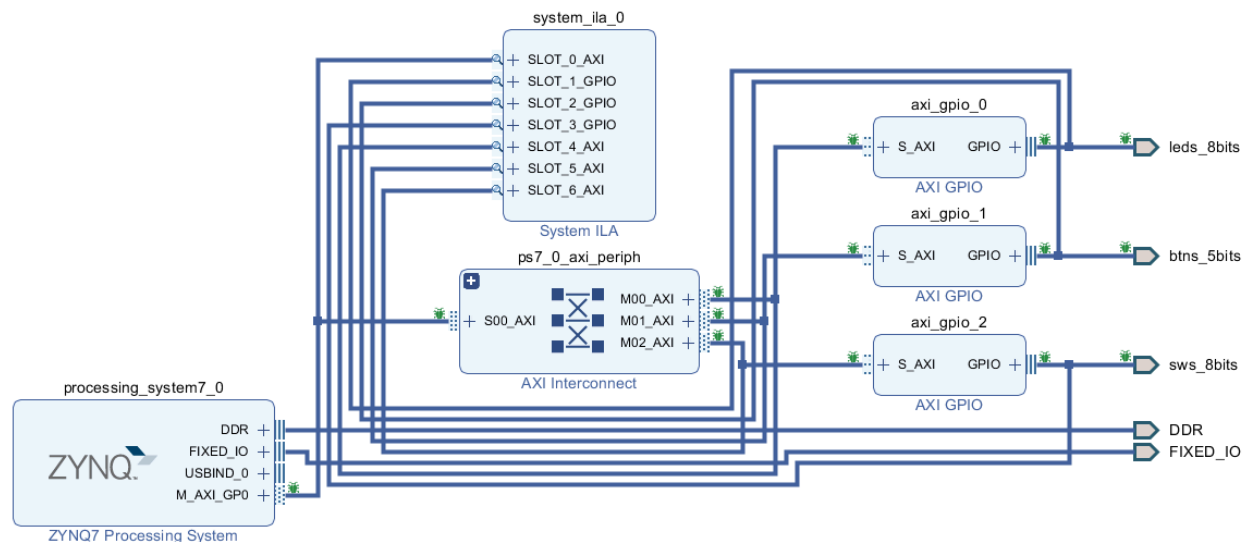


Figure 6: Block Diagram of the FPGA.

Matthews Answer:

Noting from the figure above, we can see that our LEDS, buttons, and switches are connected to each AXI GPIO, since the AXI GPIO is the bridge between our programmable logic and processing system. All of these peripherals are connected to a GPIO IP block since GPIO stands for general purpose input/output so it works for both inputs and outputs. Programmable Logic (PL) is where customized logic can be deployed to hardware-reconfigurable components. The buttons, switches, and LEDs are external signals that can be routed to signals within the PL layer.

Q: The documentation for the axi_gpio is available in the docs/IP/ directory. Based on the datasheet and the address map shown in the “Address Editor” (mentioned in instruction 7 of Step 2: Use Designer Assistance), how would you (in software) read the current state of the switches? Be specific

From Page 24 from AXI Datasheet:

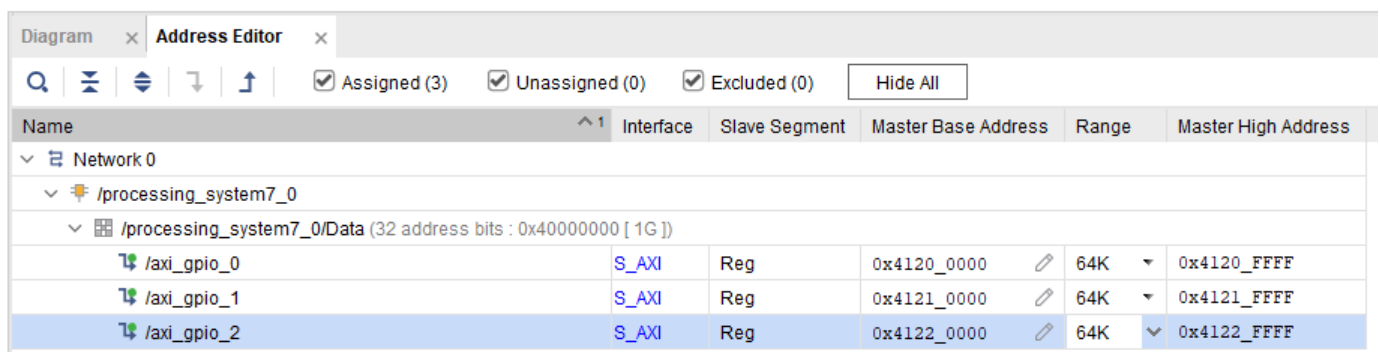
For input mode, gpio_input pins are connected to DIP switches of KC705 as follows.

- ° gpio_input(0) = GPIO_DIP_SW0
- ° gpio_input(1) = GPIO_DIP_SW1
- ° gpio_input(2) = GPIO_DIP_SW2
- ° gpio_input(3) = GPIO_DIP_SW3

You must provide stimulus as 0xA to these input pins, such as:

- ° gpio_input(0) = 0
- ° gpio_input(1) = 1
- ° gpio_input(2) = 0
- ° gpio_input(3) = 1
- On completion of a successful transaction, the GPIO_LED_7 glows.
- In case of a failure, the GPIO_LED_7 does not glow.

According to the datasheet, the GPIO pins do not have interrupts, so in order to read them, you would have to poll the data register of the GPIO. Using the Address Editor, the Master Base Address can be located; in this case, the switches are connected to axi_gpio_2, as shown in Figure 6 above. Looking at Table 2-4 in the axi_gpio documentation (Figure 7 below), the address space offset for the gpio data is 0x0000. Which means the data holding the current state of the switches will be in the axi_gpio_2 Master Base Address. With SW0 corresponding to bit 0.



Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/processing_system7_0					
/processing_system7_0/Data (32 address bits : 0x40000000 [1G])					
/axi_gpio_0	S_AXI	Reg	0x4120_0000	64K	0x4120_FFFF
/axi_gpio_1	S_AXI	Reg	0x4121_0000	64K	0x4121_FFFF
/axi_gpio_2	S_AXI	Reg	0x4122_0000	64K	0x4122_FFFF

Figure 7: Address editor for the workspace.

Table 2-4 shows the AXI GPIO registers and their addresses.

Table 2-4: Registers

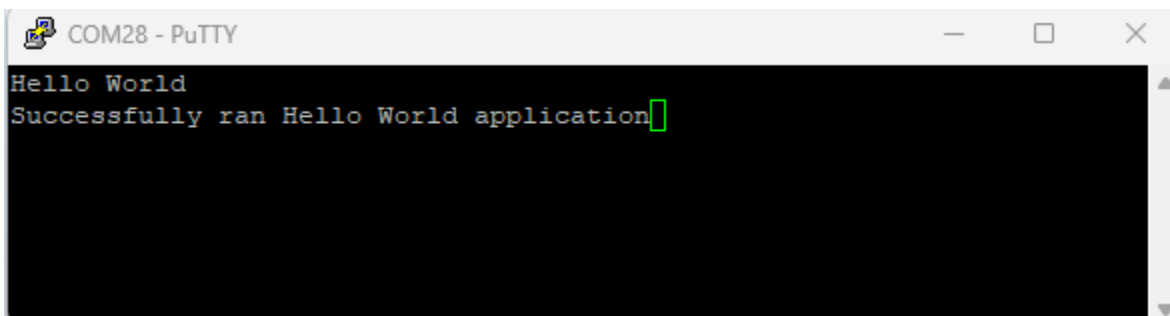
Address Space Offset ⁽³⁾	Register Name	Access Type	Default Value	Description
0x0000	GPIO_DATA	R/W	0x0	Channel 1 AXI GPIO Data Register.
0x0004	GPIO_TRI	R/W	0x0	Channel 1 AXI GPIO 3-state Control Register.
0x0008	GPIO2_DATA	R/W	0x0	Channel 2 AXI GPIO Data Register.
0x000C	GPIO2_TRI	R/W	0x0	Channel 2 AXI GPIO 3-state Control.
0x011C	GIER ⁽¹⁾	R/W	0x0	Global Interrupt Enable Register.
0x0128	IP IER ⁽¹⁾	R/W	0x0	IP Interrupt Enable Register (IP IER).
0x0120	IP ISR ⁽¹⁾	R/TOW ⁽²⁾	0x0	IP Interrupt Status Register.

Figure 8: Table 2-4 from the axi_gpio documentation.

Vitis

Q: In your writeup, use this feature and describe what `print()` does, and how. Why do you believe this function is used by Xilinx for their Hello World application, as opposed to the more conventional `printf()` function?

The main difference between `print()` and `printf()` is that in `printf()`, the 'f' stands for 'format'. The `printf()` function formats the string before sending the data, while `print()` sends the string character by character. Since `print()` does not format, it runs faster and uses less memory than `printf()`, which is one reason Xilinx might prefer to use `print()` for their Hello World application.



```
COM28 - PuTTY
Hello World
Successfully ran Hello World application
```

Figure 9: Address editor for the workspace.

Q : i) Take a screen capture of an LED wire turning on? Can you turn the LED on and off fast enough to get a screen capture of the Logic Analyzer displaying this pulse? If so, then provide this screen capture as well. For how long does the pulse stay high?

Within the Hardware Manager we set a trigger on mode “R” to look at the rising edge of SLOT_2_GPIO_tri_i[4:0] which was a probe set in the ILA. We can see when I set this trigger and trying to quickly switch the Switch button we see it turn on and stay on for 500ms. We were unable to achieve a shorter pulse with the mode we were in.

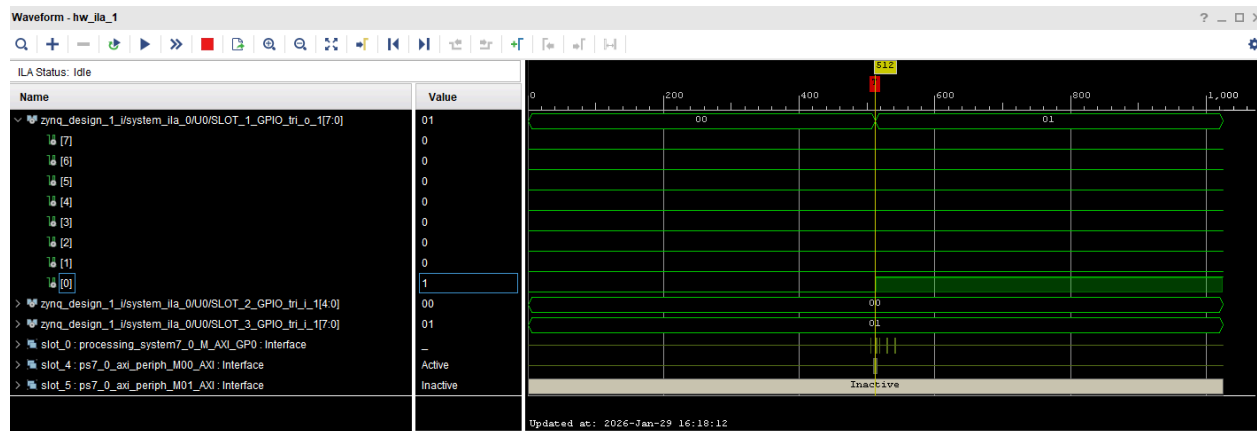
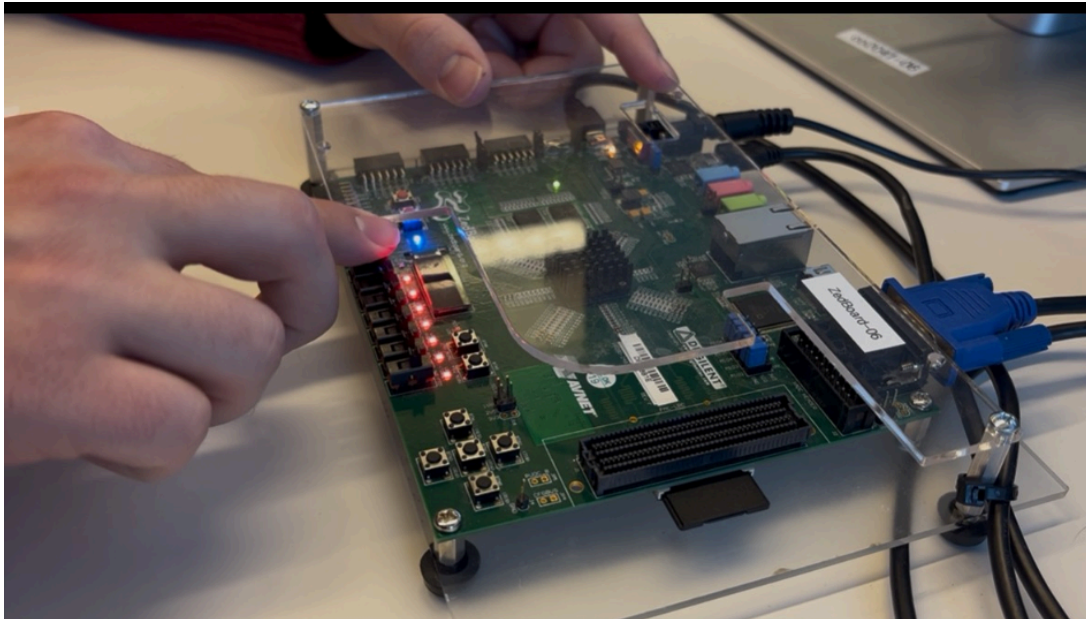
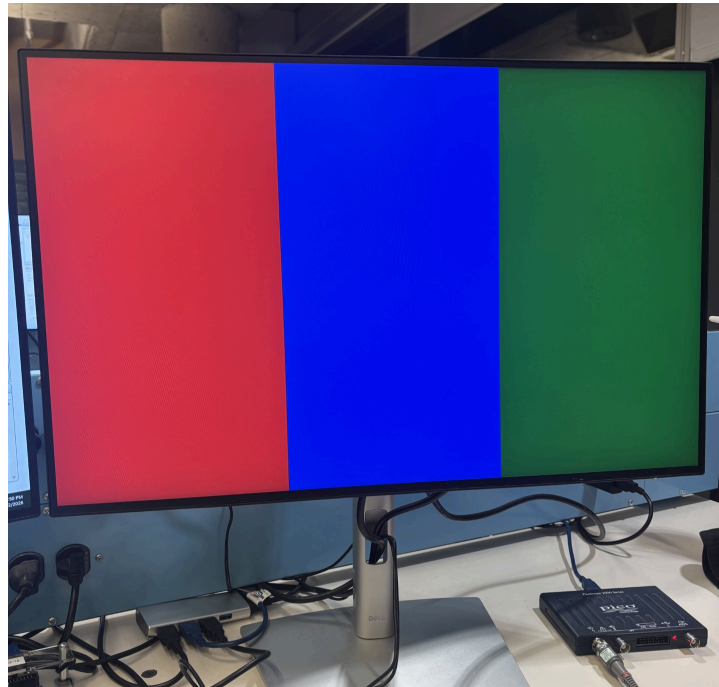


Figure 10: Address editor for the workspace.

Q : Modify the hello_world application to also interface with the switches, buttons, and LEDs that are configured in the programmable logic. For example, have the application print out the state of the switches when a button is pressed, or light up certain LEDs given an input integer.



Q : In VIVADO, add these peripherals to your project , connect and then configure them to generate a 640x480 output signal



Q : Modify the configuration registers for correct VDMA operation, and in your writeup, provide a justification based on the VDMA documentation for how you set these values:

For the control register configuration, we wrote 0x03 after reviewing the MM2S_VDMACR Register Details. Bit 0 enables the control register, and bit 1 enables Circular Mode.

For the frame buffer offset, we wrote 0x00 because we did not want any offset to our address for the test image

For the Start Address Offset, we set the value to be the address of the first element of the test_image array.

For the frame delay stride and the hsize, we set the values to 1280, which is double the 640 height because each pixel is 2 Bytes

The VSize is 480

Q :In your writeup, explain how you converted these color values valid values for the 16-bit framebuffer.

With the given values being 24 bit colors, they needed to be shortened into a 12bit color. It can be inferred that each color must have 8 bits within the 24 bits. We considered the MSB from each color to be the most important. So, we took bits 23-20 ,15-12, and 7-4, and used those as our 12 bit color.

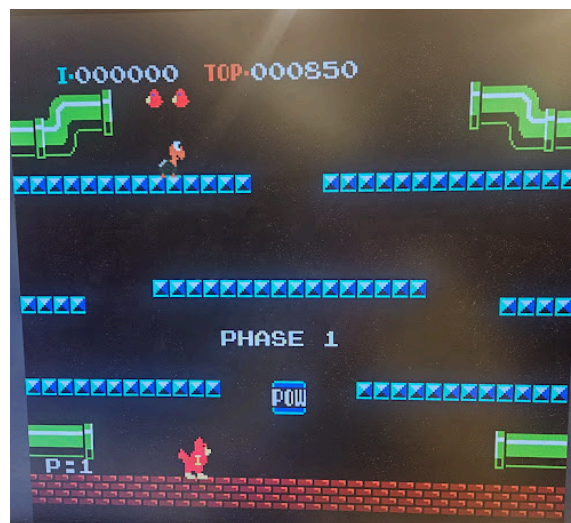
Q :Modify the nes_bootloader code such that the NES games are reasonably playable. In your report, describe your general approach to implementing both of the NESCore_Callback functions.

To get the output frame working with our monitor over VGA we first need to scale the output. The default output for the NES emulator is 256x240 and we need 640x480. We can easily just scale the height by 2x to get a 480 pixel tall output but the width is more complicated. Since we can't easily map the width by the same scaling factor, we need to center the screen and have "black bars" flanking the image. We will scale the height and width by 2x and then shift the video right by $(640 - 256*2) / 2 = 64$.

To get button inputs working to control the NES game, we needed to read from the GPIO address associated with the buttons/switches we want to use to control. We do this using this code:

```
Xil_In32(XPAR_AXI_GPIO_X_BASEADDR);
```

Then we check to see if the value we read includes the bit associated with each button. If we are holding up, then `*pdwPad1 |= NCTL_UP`, etc.



Bonus Credit

- Interactive menu for selecting NES games, either a nice text-based menu (**5 bonus points**) or a graphical menu (10 bonus points).

For the menu, we used putty to prompt the user to select a game based on its address in a string array. We created a string array that has every .nes game for the user to choose from. We then use print to give the options to the user. We then wait for the user to input a choice (we check that it is a valid choice). We store the user's choice and launch the game.

- A method for exiting a current game and returning back to the menu (**5 bonus points**).

Continuing from the PuTTY implementation of a game select, we allow the user to press 'q' to quit the game. After the game is terminated, it prompts the user again to choose a game.

- Interfacing with an original NES controller (**10 bonus points**)

The hardest bonus point was getting the SNES controller to connect to the FPGA. We did so by first learning how the SNES controller outputs its button states and how it latches the values to send the data one at a time. Once we learned this, we created VHDL code that reads that data in and combines it into one 16 bit data value. We added this VHDL script as an IP block into our Vivado design which allowed us to connect the PMOD peripheral to the VHDL code and then the 16 bit output to a GPIO for our processor to read. From there it was no different then using the button GPIO.

This section gave us a significant headache because we did not know that you had to manually refresh an IP block when you changed its underlying VHDL. We were chasing errors that we fixed and were just caused by our block being out of data (Vivado does not tell you about this).

Student Effort

The following represents the percentage of effort from each team member for MP-0:

Cameron Gilbertson:	25%
Matthew Smith:	25%
Jessica Reiff:	25%
Collin Van Weelden:	25%